United International University

Department of Computer Science and Engineering

Structured Programming Laboratory — Fall 2026

Topic: Array Related Problems in C — Duration: 2.5 hours

Lab Sheet: Array Related Problems

Prepared by: Ashraful Islam Paran, Dept. of CSE, UIU. Follow instructions carefully and complete all assignments within the designated slot.

1. Objectives

By the end of this lab, students will be able to:

- Declare, initialize, read, and display one-dimensional arrays.
- Traverse arrays using loops and access elements through valid indices.
- Perform searching, counting, summation, averaging, and reversal.
- Insert, delete, replace, copy, and rearrange array elements.
- Apply elementary sorting and duplicate-removal techniques.
- Perform set operations such as intersection, union, and difference.

2. Introduction to Arrays in C

An array stores multiple values of the same data type in contiguous memory locations. In C, array indexing starts at 0. Therefore, an array containing n elements has valid indices from 0 to $n - 1$.

Important notes

- Declare an array with a sufficient capacity before storing elements.
- Use a loop to read or process all elements of an array.
- For an array of size `n`, never access an index outside the range `0` to `n-1`.
- In the insertion problem, the specified position is a zero-based index; inserting at position `n` appends the value.
- In the deletion problem, the position must be a valid existing index from `0` to `n-1`.
- The solutions below use a maximum capacity of 1000 elements. Inputs are assumed to satisfy the stated size constraints.
- Set-operation outputs contain each value only once and preserve the order in which values are first encountered.

3. Print an Array in Reverse Order (*)

Task: Read n integers into an array and print them from the last valid index down to index 0. Do not modify the original array.

Sample input

```
5
1 2 3 4 5
```

Sample output

```
5 4 3 2 1
```

array_reverse_print.c

```c
#include <stdio.h>

int main(void) {
    int n, a[1000];

    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (int i = n - 1; i >= 0; i--)
        printf("%d ", a[i]);

    printf("\n");
    return 0;
}
```

4. Sum of All Array Elements (*)

Task: Read n integers into an array and calculate the sum of all elements.

Sample input

```
5
1 2 3 4 5
```

Sample output

```
15
```

array_sum.c

```c
#include <stdio.h>

int main(void) {
    int n, a[1000], sum = 0;

    scanf("%d", &n);
```

```c
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        sum += a[i];
    }

    printf("%d\n", sum);
    return 0;
}
```

5. Sum of Even Elements (*)

Task: Read n integers into an array and calculate the sum of the even-valued elements.

Sample input

```
6
2 8 3 9 0 1
```

Sample output

```
10
```

sum_even_elements.c

```c
#include <stdio.h>

int main(void) {
    int n, a[1000], sum = 0;

    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (int i = 0; i < n; i++)
        if (a[i] % 2 == 0)
            sum += a[i];

    printf("%d\n", sum);
    return 0;
}
```

6. Average of Floating-Point Array Elements (*)

Task: Read n floating-point numbers into an array and calculate their average. Print the result with two digits after the decimal point.

Sample input

```
5
1.2 5.6 10.3 4.5 5.2
```

Sample output

```
5.36
```

array_average.c

```c
#include <stdio.h>

int main(void) {
    int n;
    double a[1000], sum = 0.0;

    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        scanf("%lf", &a[i]);
        sum += a[i];
    }

    if (n > 0)
        printf("%.2f\n", sum / n);

    return 0;
}
```

7. Sum of Elements at Even Indices (*)

Task: Read n integers into an array and calculate the sum of the elements stored at even indices (0, 2, 4, ...).

Sample input

```
5
1 2 3 4 5
```

Sample output

```
9
```

sum_even_indices.c

```c
#include <stdio.h>

int main(void) {
```

```c
    int n, a[1000], sum = 0;

    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (int i = 0; i < n; i += 2)
        sum += a[i];

    printf("%d\n", sum);
    return 0;
}
```

8. Add Corresponding Elements of Two Arrays (**)

Task: Read n integers into array A and another n integers into array B. Store the sum of corresponding elements in a third array C, then print C.

Sample input

```
5
1 2 3 4 5
2 8 3 4 8
```

Sample output

```
3 10 6 8 13
```

add_two_arrays.c

```c
#include <stdio.h>

int main(void) {
    int n, a[1000], b[1000], c[1000];

    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    for (int i = 0; i < n; i++)
        scanf("%d", &b[i]);

    for (int i = 0; i < n; i++)
        c[i] = a[i] + b[i];

    for (int i = 0; i < n; i++)
        printf("%d ", c[i]);

    printf("\n");
    return 0;
}
```

9. Reverse the Elements Within an Array (**)

Task: Read n integers into an array, reverse the elements within the same array, and print the modified array from index 0 to the last valid index.

Sample input

```
5
1 2 3 4 5
```

Sample output

```
5 4 3 2 1
```

reverse_array_in_place.c

```c
#include <stdio.h>

int main(void) {
    int n, a[1000], temp;

    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (int i = 0; i < n / 2; i++) {
        temp = a[i];
        a[i] = a[n - 1 - i];
        a[n - 1 - i] = temp;
    }

    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n");
    return 0;
}
```

10. Find Maximum and Minimum with Their Indices (**)

Task: Read n integers and find the maximum and minimum values, along with their index positions. If a value occurs more than once, report the first occurrence.

Sample input

```
6
2 8 3 9 0 1
```

Sample output

```
Max: 9, Index: 3
Min: 0, Index: 4
```

maximum_minimum.c

```c
#include <stdio.h>

int main(void) {
    int n, a[1000], maxIndex = 0, minIndex = 0;

    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (int i = 1; i < n; i++) {
        if (a[i] > a[maxIndex])
            maxIndex = i;
        if (a[i] < a[minIndex])
            minIndex = i;
    }

    if (n > 0) {
        printf("Max: %d, Index: %d\n", a[maxIndex], maxIndex);
        printf("Min: %d, Index: %d\n", a[minIndex], minIndex);
    }

    return 0;
}
```

11. Count Vowels in a Character Array (*)

Task: Read n alphabetic characters into a character array and count the vowels (A, E, I, O, U), considering both uppercase and lowercase letters.

Sample input

```
7
AKIOUEH
```

Sample output

```
Count: 5
```

count_vowels.c

```c
#include <stdio.h>
#include <ctype.h>

int main(void) {
    int n, count = 0;
    char a[1000];

    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf(" %c", &a[i]);
```

```
    for (int i = 0; i < n; i++) {
        char ch = (char)tolower((unsigned char)a[i]);
        if (ch == 'a' || ch == 'e' || ch == 'i' ||
            ch == 'o' || ch == 'u')
            count++;
    }

    printf("Count: %d\n", count);
    return 0;
}
```

12. Search for a Number in an Array (*)

Task: Read n integers into an array and then read a target number. If the target is found, print all its index positions. Otherwise, print `NOT FOUND`.

Sample input

```
8
7 8 1 3 2 6 4 3
3
```

Sample output

```
FOUND at index position: 3 7
```

linear_search.c

```c
#include <stdio.h>

int main(void) {
    int n, a[1000], target, found = 0;

    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    scanf("%d", &target);

    for (int i = 0; i < n; i++) {
        if (a[i] == target) {
            if (!found)
                printf("FOUND at index position: ");
            printf("%d ", i);
            found = 1;
        }
    }

    if (!found)
        printf("NOT FOUND");
```

```
    printf("\n");
    return 0;
}
```

13. Copy an Array in Reverse Order (*)

Task: Read n integers into array A. Copy the elements of A into array B in reverse order. Finally, display both arrays.

Sample input

```
5
7 8 1 3 2
```

Sample output

```
Array A: 7 8 1 3 2
Array B: 2 3 1 8 7
```

copy_reverse.c

```c
#include <stdio.h>

int main(void) {
    int n, a[1000], b[1000];

    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (int i = 0; i < n; i++)
        b[i] = a[n - 1 - i];

    printf("Array A: ");
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\nArray B: ");
    for (int i = 0; i < n; i++)
        printf("%d ", b[i]);

    printf("\n");
    return 0;
}
```

14. Insert an Element at a Specified Position (**)

Task: Read n integers into an array. Then read a number and a zero-based position. Insert the number at that position by shifting the required elements to the right.

Sample input

```
5
32 14 9 48 6
16 0
```

Sample output

```
16 32 14 9 48 6
```

array_insertion.c

```c
#include <stdio.h>

int main(void) {
    int n, a[1001], number, position;

    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    scanf("%d %d", &number, &position);

    if (position >= 0 && position <= n && n < 1000) {
        for (int i = n; i > position; i--)
            a[i] = a[i - 1];

        a[position] = number;
        n++;

        for (int i = 0; i < n; i++)
            printf("%d ", a[i]);
        printf("\n");
    } else {
        printf("Invalid position or array size\n");
    }

    return 0;
}
```

15. Delete an Element from a Specified Position (*)

Task: Read n integers into an array. Then read a zero-based position and delete the element at that position by shifting the following elements to the left.

Sample input

```
5
32 14 9 48 6
0
```

Sample output

```
14 9 48 6
```

array_deletion.c

```c
#include <stdio.h>

int main(void) {
    int n, a[1000], position;

    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    scanf("%d", &position);

    if (position >= 0 && position < n) {
        for (int i = position; i < n - 1; i++)
            a[i] = a[i + 1];

        n--;

        for (int i = 0; i < n; i++)
            printf("%d ", a[i]);
        printf("\n");
    } else {
        printf("Invalid position\n");
    }

    return 0;
}
```

16. Swap the Contents of Two Arrays (**)

Task: Read n integers into array A and m integers into array B. Swap the complete contents of the arrays, including their logical sizes, and then display both arrays. The arrays may have different sizes.

Sample input

```
3
7 8 1
5
3 2 1 9 4
```

Sample output

```
Array A: 3 2 1 9 4
Array B: 7 8 1
```

swap_two_arrays.c

```c
#include <stdio.h>

int main(void) {
    int n, m, a[1000], b[1000], temp[1000], tempSize;

    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    scanf("%d", &m);
    for (int i = 0; i < m; i++)
        scanf("%d", &b[i]);

    for (int i = 0; i < n; i++)
        temp[i] = a[i];
    tempSize = n;

    for (int i = 0; i < m; i++)
        a[i] = b[i];
    n = m;

    for (int i = 0; i < tempSize; i++)
        b[i] = temp[i];
    m = tempSize;

    printf("Array A: ");
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\nArray B: ");
    for (int i = 0; i < m; i++)
        printf("%d ", b[i]);
```

```
    printf("\n");
    return 0;
}
```

17. Replace Multiples of 3 with -1 (*)

Task: Read n positive integers into array A. Replace every element divisible by 3 with -1, then display the modified array.

Sample input

```
8
7 8 1 3 2 6 4 3
```

Sample output

```
7 8 1 -1 2 -1 4 -1
```

replace_multiples_of_three.c

```c
#include <stdio.h>

int main(void) {
    int n, a[1000];

    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (int i = 0; i < n; i++)
        if (a[i] % 3 == 0)
            a[i] = -1;

    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n");
    return 0;
}
```

18. Replace Elements at Odd Indices with 0 (*)

Task: Read n positive integers into array A. Replace all elements whose indices are odd (1, 3, 5, ...) with 0. Display the modified array.

Sample input

```
8
7 8 1 3 2 6 4 3
```

Sample output

```
7 0 1 0 2 0 4 0
```

replace_odd_indices.c

```c
#include <stdio.h>

int main(void) {
    int n, a[1000];

    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (int i = 1; i < n; i += 2)
        a[i] = 0;

    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n");
    return 0;
}
```

19. Sort an Array in Ascending Order (Bubble Sort) (***)

Task: Read n integers into array A and sort them in ascending order using the Bubble Sort algorithm. Finally, display the sorted array.

Sample input

```
8
7 8 1 3 2 6 4 3
```

Sample output

```
1 2 3 3 4 6 7 8
```

bubble_sort.c

```c
#include <stdio.h>

int main(void) {
    int n, a[1000], temp;

    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j];
```

```
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }

    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n");
    return 0;
}
```

Keep Coding!

Strong fundamentals in arrays are essential for structured programming, data organization, and problem solving.